

void preencher_pilha

Esta função vai preencher uma pilha com cartas. Faz um ciclo por quantas cartas tem a pilha e iguala a do deck à da pilha para a copiar. (*c_idx)++ é importante para também avançar a carta no deck.

void distribuir_cartas_deck

Esta função prepara o jogo imprimindo também o que é que está a fazer. Tínhamos bugs ao iniciar o jogo então colocamos os printf's para perceber onde é que falhava. Primeiro carregamos a paciência. Depois criamos o baralho e baralhamos as cartas. Depois distribuímos as cartas e como elas já vão estar nas pilhas, já não precisamos do deck então podemos libertar a memória que ele ocupa. Retorna 1 em caso de sucesso

void efetuar_movimento

Esta função vai criar a cópia do estado e depois executar o movimento do jogador. Guarda numa variável chamada msg a mensagem de que o movimento foi realizado com sucesso.

void processar_jogada

Esta função vai processar a jogada. Primeiro pede a jogada ao jogador. Se as pilhas não existirem guarda o erro de que as pilhas não existem em msg. Depois vai encontrar as regras para a movimentação entre aquelas pilhas e se ele for válido chama a função anterior para processar a jogada, caso contrário guarda em msg o erro.

int tentar_auto_movimento

Esta função faz parte dos movimentos automáticos que alguns jogos podem ter. Primeiro faz um ciclo por todas as regras do automático e confirma se a origem que esta regra existe é igual ao nome da pilha de onde quero tirar a carta e se o destino que esta regra exige é o nome da pilha de destino¹. Tem de ser simultaneamente verdadeiro para a origem e para o destino.

Quando encontrar a regra correta vai buscar as flags e tiver +, apenas se a pilha tiver 13 cartas ou mais é que tenta mover as 13 para o destino. Se o movimento retornar válido, move as cartas e retorna 1. Se não tiver + verifica se pode efetuar o movimento de uma carta

¹ Exemplo: A função quer mover da pilha (TAB) para a fundação (FUND). Se a regra que sair for para mover do STOCK para TAB reprova. Apenas quando a regra for a de mover do TAB para FUND é que encontramos a regra certa

void executar_movimentos_automaticos

Esta função vai verificar se existem movimentos automáticos para fazer e se houver, fá-los. O primeiro ciclo while vai funcionar enquanto conseguir encontrar movimentos automáticos para fazer. No segundo while, ele vai testar para todas as colunas se pode fazer movimentos. Temos o moveu == 0 para que mal ele faça um movimento esse ciclo termine. Neste ciclo, para todas as pilhas não vazias, vai fazer outro para todas as pilhas de destino e testar os movimentos. Caso consiga mover muda moveu para 1 e o ciclo de procura para aquela coluna de destino e origem para e recomeça o ciclo pois o tabuleiro mudou.

void tratar_movimento

Esta função auxiliar simplesmente trata do movimento onde processa a jogada do jogador e depois verifica os automáticos e faz reset às dicas.

void tratar_undo

Esta função faz o undo e guarda a mensagem de sucesso em msg e faz reset às dicas

void tratar_save

Esta função é responsável por guardar o jogo. Chama a função que guarda e guarda a mensagem de erro ou sucesso em msg.

void executar_comando_dica

Esta função é invocada quando o jogador pede uma dica e executa a função de pedir dica alterando os valores para indicar a dica e guarda a mensagem de aviso ou de erro.

void executar_comando_jogador

Esta função funciona invocando as anteriores em função do input do jogador.

void verificar_estado_vitoria

Esta função, caso haja vitória vai limpar o ecrã, imprimir a vitória e parar o jogo alterando o a_jogar.

void impressao

Esta função é a principal que imprime o tabuleiro de jogo após cada jogada. Primeiro começa por imprimir a mensagem caso haja, e depois apaga a mesma da variável. Depois mostra o tabuleiro atual e imprime o menu de escolhas e recebe a jogada do jogador e executa o comando. Temos o else para caso seja posta uma letra o computador não entre num bug infinito. Ele funciona pegando em carácter a carácter até este ser diferente do enter ('\\n') ou chegar ao fim do ficheiro (EOF) que pode acontecer caso o utilizador tenha usado um atalho para forçar o terminal a parar. Este ciclo apenas tem ; pois o getch limpa logo do buffer o carácter. Depois guarda a mensagem de erro. Se executar o movimento verifica se há vitória.

void iniciar_loop_jogo

Esta função começa o loop de jogo e enquanto a_jogar se mantiver igual a 1, o jogo está sempre a funcionar.

EstadoJogo* inicializar_estado

Esta função é chamada quando o jogo está a ser carregado para poder ser alocada a memória necessária e meter todos os contadores a 0 e regras, pilhas, etc em NULL retornando o endereço.

int main

A função que vai fazer tudo funcionar. Cria um array onde vai estar o caminho para o jogo. Depois inicia a semente para usar no random quando estamos a baralhar as cartas. Depois inicia o estado. Depois ao invocar menuPrincipal vai imprimir as opções e receber uma opção do jogador. Se ele quiser jogar (2) e conseguir carregar o jogo vai iniciar o loop de jogo. Se selecionar carregar save (1) e conseguir carregar o jogo, inicia o loop já com o save carregado. Se depois for diferente de 0, é porque houve um erro a carregar o jogo pois se for 0 imprime a mensagem de despedida e retorna 0 para terminar o programa.